Sri Sivasubramaniya Nadar College of Engineering,

Kalavakkam - 603 110

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Course Code	UIT2702	Course Name	Cloud and Distributed Computing
Regulation	2021	Academic Year	**2026-2027**
Semester		VII	
Department Offering the Course		Information Technology	
Exercise: 01		Inter-Process Communication in Distributed Computing	

Name: Rahul Akshath Rajendran

Reg No: 3122235002095

Sem / Sec: VII B

Exp. No: 01 **Date: 08-07-2026**

INTER-PROCESS COMMUNICATION IN DISTRIBUTED COMPUTING

AIM

To implement and understand inter-process communication (IPC) using sockets in a distributed client-server setup, and to exchange both plain text messages and serialized Python objects between two processes.

QUESTION

Task 1 - Setup Basic Client-Server Communication:

- Create a simple socket connection where the client sends a message and server responds.
- On server side, bind to a host and port, listen for client connections, and accept messages.

- Implement message handling where the server parses and sums numbers.

Task 2 - Enhanced Communication with Object Serialization:

- Define a custom data object (DataObject) with attributes (name and values) and enable object-based data transfer.
- Serialize and deserialize objects using Python's pickle module.
- Send a DataObject from client to server and compute the sum of values on the server.
- Send a response from the server back to the client, indicating the sum of values.

ALGORITHM / STEPS

Task 1 - Server

1. Import the socket module and create a TCP socket using socket.socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the host "localhost" and the port 5000.
3. Call listen() so that the socket starts accepting client connections.
4. Accept a connection; this returns a new socket for that client and the client's address.
5. Receive the message from the client using recv() and decode it into a string.
6. Split the string on spaces, convert every token into an integer and store them in a list.
7. Compute the sum of the list using sum().
8. Encode the result and send it back to the client.
9. Close the client socket and then the server socket.

Task 1 - Client

1. Create a TCP socket.
2. Connect to the server running at ("localhost", 5000).
3. Read the numbers from the user as one space separated string.
4. Encode the string and send it to the server.
5. Receive the reply from the server and decode it.

6. Display the sum returned by the server.

7. Close the socket.

Task 2 - Client

1. Import socket and pickle.

2. Define the class DataObject with the attributes name and values.

3. Create an object of DataObject, for example DataObject("Student", [10, 20, 30, 40]).

4. Serialize (marshal) the object into a byte stream using pickle.dumps().

5. Create a TCP socket and connect to the server at ("localhost", 5001).

6. Send the serialized bytes to the server.

7. Receive the reply from the server, decode it and display it.

8. Close the socket.

Task 2 - Server

1. Import socket and pickle.

2. Define the same DataObject class, because pickle needs the class definition to rebuild theobject.

3. Create a TCP socket, bind it to ("localhost", 5001) and listen for a client.

4. Accept the connection and receive the byte stream using recv(4096).

5. Deserialize (unmarshal) the byte stream back into an object using pickle.loads().

6. Print the name and values attributes of the received object.

7. Compute the sum of the values list.

8. Build the response string "Sum of values = <total>", encode it and send it to the client.

9. Close the client socket and the server socket.

CODE

Task 1 - Basic Client-Server Communication

server.py

```
import socket
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(("localhost", 5000))
server.listen(1)

print("Server is waiting for connection...")

client, address = server.accept()
print("Connected to:", address)

message = client.recv(1024).decode()
print("Received:", message)

numbers = list(map(int, message.split()))

total = sum(numbers)
client.send(str(total).encode())

client.close()
server.close()
```

client.py

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect(("localhost", 5000))

message = input("Enter numbers separated by space: ")

client.send(message.encode())
result = client.recv(1024).decode()

print("Sum from server:", result)

client.close()
```

Task 2 - Communication using Object Serialization (pickle)

server_pickle.py

```
import socket
import pickle

# Custom class
class DataObject:
    def __init__(self, name, values):
        self.name = name
        self.values = values

# Create socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(("localhost", 5001))
server.listen(1)

print("Server waiting...")

client, address = server.accept()
print("Connected to:", address)

# Receive serialized object
data = client.recv(4096)
```

```
# Deserialize object
obj = pickle.loads(data)

print("Name:", obj.name)
print("Values:", obj.values)

# Calculate sum
total = sum(obj.values)

# Send response
response = f"Sum of values = {total}"
client.send(response.encode())

client.close()
server.close()
```

client_pickle.py

```
import socket
import pickle

class DataObject:
    def __init__(self, name, values):
        self.name = name
        self.values = values

obj = DataObject("Student", [10, 20, 30, 40])
data = pickle.dumps(obj)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 5001))
client.send(data)

response = client.recv(1024).decode()
print(response)
```

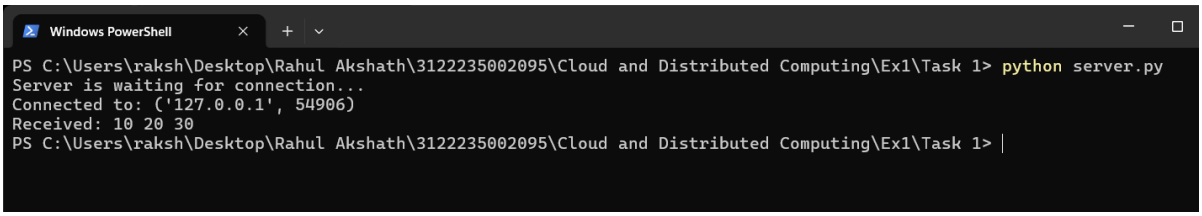
client.close()

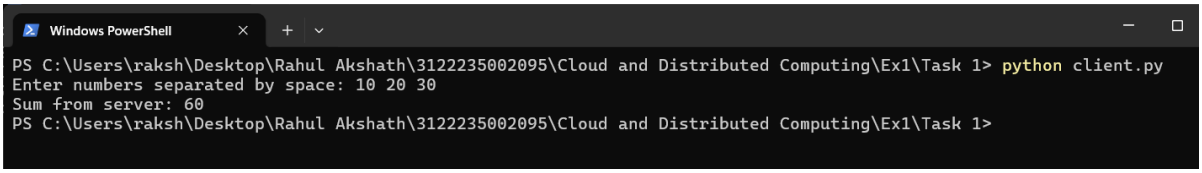
Steps to execute

- Task 1: open two terminals. Run "python3 server.py" in the first terminal and "python3 client.py" in the second terminal, then type the numbers in the client.
- Task 2: open two terminals. Run "python3 server_pickle.py" first and then "python3 client_pickle.py".

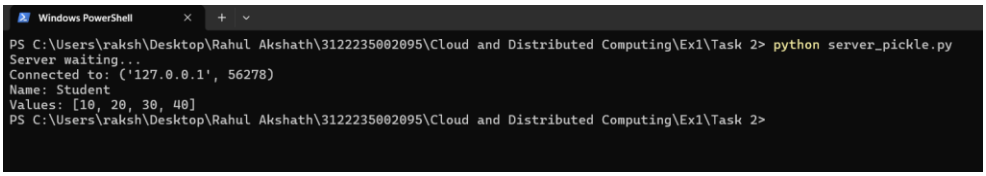
OUTPUT SCREENSHOTS

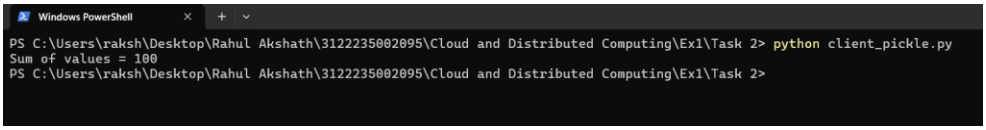
Task 1





Task 2





CONCLUSION

Inter-process communication between two independent processes was implemented successfully using TCP sockets. In Task 1 the client sent a plain text

message and the server parsed the numbers, computed their sum and returned the result, which shows the basic request-response model of the client-server architecture. In Task 2 the communication was extended to structured data: a user defined DataObject was serialized with the pickle module, transmitted as a byte stream over the socket and reconstructed on the server, where the sum of its values was computed and sent back. Thus the need for marshalling and unmarshalling of objects in distributed systems, and the working of socket based IPC, were understood and verified.